

미분, 최적화 기초

머신러닝 모델을 학습시키는 과정은 본질적으로 주어진 데이터에 가장 잘 맞는 모델 파라미터를 찾는 최적화 문제입니다. 이러한 최적화 문제를 해결하기 위해서는 함수의 변화율을 이해하는 미분(Calculus)과 효율적으로 최적점을 찾아가는 수치 최적화(Numerical Optimization) 기법에 대한 깊이 있는 이해가 필수적입니다. 본 장에서는 다변수 함수의 미분 개념부터 시작하여, 머신러닝에서 가장 널리 사용되는 최적화 알고리즘인 경사하강법과 뉴턴법의 원리를 탐구합니다.

3.1 다변수 미분, 그래디언트, 야코비안, 헤시안

머신러닝 모델의 손실 함수는 대부분 여러 개의 파라미터(변수)를 가지는 다변수 함수입니다. 이러한 함수의 최적점을 찾기 위해서는 각 변수에 대한 함수의 변화율을 알아야 합니다.

그래디언트 (Gradient, ∇f)

직관적 이해: 다변수 함수에서 그래디언트는 함수 값이 가장 빠르게 증가하는 방향과 그 증가율을 나타내는 벡터입니다. 마치 산을 오를 때 가장 가파른 오르막길을 가리키는 방향과 같습니다. 최적화에서는 이 그래디언트의 반대 방향으로 이동하여 함수 값을 줄여나갑니다.

수학적 의미: 스칼라 값 함수 $f(x_1, x_2, \dots, x_n)$ 의 그래디언트는 각 변수에 대한 편미분(partial derivative)을 원소로 하는 벡터입니다.
$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

예제: 주어진 $f(x,y)$ 의 ∇f 계산 함수 $f(x, y) = x^2 + 3xy + y^3$ 의 그래디언트를 계산해 봅시다.
$$\frac{\partial f}{\partial x} = 2x + 3y \quad \frac{\partial f}{\partial y} = 3x + 3y^2$$
 따라서, $\nabla f(x, y) = \begin{pmatrix} 2x + 3y \\ 3x + 3y^2 \end{pmatrix}$ 입니다.

야코비안 (Jacobian, J_f)

직관적 이해: 야코비안은 여러 개의 입력 변수를 받아 여러 개의 출력 변수를 내놓는 함수(벡터-값 함수)의 '변화율 행렬'입니다. 각 출력 변수가 각 입력 변수에 대해 얼마나 변하는지를 나타냅니다.

수학적 의미: 벡터-값 함수 $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x}))^T$ (여기서 $\mathbf{x} = (x_1, \dots, x_n)^T$)의 야코비안 행렬은 각 x_i 를 각 x_j 에 대해 편미분한 값들을 원소로 가집니다.
$$J_{\mathbf{f}}(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}$$
 스칼라 함수 $f(\mathbf{x})$ 의 경우, 야코비안은 그래디언트 벡터의 전치($\nabla f(\mathbf{x})^T$)와 같습니다.

체인룰 (Chain Rule)과 야코비안: 복합 함수의 미분에서 체인룰은 야코비안 행렬의 곱셈으로 표현될 수 있습니다. 이는 역전파(Backpropagation) 알고리즘의 핵심 원리입니다.

헤시안 (Hessian, H_f 또는 $\nabla^2 f$)

직관적 이해: 헤시안 행렬은 함수의 '곡률(curvature)'에 대한 정보를 담고 있습니다. 1차 미분(그래디언트)이 함수의 기울기를 알려준다면, 2차 미분(헤시안)은 기울기가 얼마나 빠르게 변하는지, 즉 함수가 얼마나 볼록하거나

나 오목한지를 알려줍니다. 이는 최적화에서 극소점, 극대점, 안장점 등을 판단하는 데 사용됩니다.

수학적 의미: 스칼라 값 함수 $f(x_1, x_2, \dots, x_n)$ 의 헤시안 행렬은 모든 2차 편미분으로 구성된 대칭 행렬입니다.
$$H_f(\mathbf{x}) = \nabla^2 f(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$
 함수가 연속적인 2차 편미분을 가질 경우, 교차 편미분 $\frac{\partial^2 f}{\partial x_i \partial x_j}$ 와 $\frac{\partial^2 f}{\partial x_j \partial x_i}$ 은 같으므로 헤시안 행렬은 항상 대칭 행렬입니다.

헤시안의 양의 정부호성 (Positive Definiteness): 최적화에서 헤시안 행렬의 특성은 함수의 극점을 판별하는 데 중요합니다.

- **양의 정부호 (Positive Definite):** 헤시안 행렬이 양의 정부호이면 해당 지점은 **극소점**입니다. (모든 고유값이 양수)
- **음의 정부호 (Negative Definite):** 헤시안 행렬이 음의 정부호이면 해당 지점은 **극대점**입니다. (모든 고유값이 음수)
- **부정부호 (Indefinite):** 헤시안 행렬이 부정부호이면 해당 지점은 **안장점**입니다. (양수와 음수 고유값이 모두 존재)
- **양의 준정부호 (Positive Semi-definite):** 극소점일 가능성이 있지만, 추가적인 분석이 필요합니다. (모든 고유값이 0 이상)

예제: 주어진 $f(x, y)$ 의 H_f 계산 함수 $f(x, y) = x^2 + 3xy + y^3$ 의 헤시안 행렬을 계산해 봅시다. 먼저 1차 편미분은 다음과 같습니다. $\frac{\partial f}{\partial x} = 2x + 3y$, $\frac{\partial f}{\partial y} = 3x + 3y^2$ 이제 2차 편미분을 계산합니다. $\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x}(2x + 3y) = 2$, $\frac{\partial^2 f}{\partial y^2} = \frac{\partial}{\partial y}(3x + 3y^2) = 6y$, $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial}{\partial y}(2x + 3y) = 3$, $\frac{\partial^2 f}{\partial y \partial x} = \frac{\partial}{\partial x}(3x + 3y^2) = 3$ 따라서, $H_f(x, y) = \begin{pmatrix} 2 & 3 \\ 3 & 6y \end{pmatrix}$ 입니다.

2변수 이차함수의 등고선과 최저점 사례 이차 함수 $F(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} + \mathbf{d}^T \mathbf{x} + c$ 의 형태는 헤시안 행렬 A 의 고유값과 고유벡터에 의해 결정됩니다. 헤시안 행렬의 고유값은 함수의 곡률을 나타내며, 고유벡터는 곡률이 가장 큰(또는 작은) 방향을 나타내는 주축(principal axes)이 됩니다.

예를 들어, $F(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2$ 와 같은 이차 함수의 등고선은 타원 형태를 띠며, 이 타원의 주축은 헤시안 행렬의 고유벡터 방향과 일치합니다. 이 함수의 헤시안 행렬은 $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ 이고, 고유값은 $\lambda_1=1, \lambda_2=3$ (모두 양수)입니다. 이는 함수가 아래로 볼록하며 명확한 최저점을 가진다는 것을 의미합니다.

고유값 (λ_i)	함수의 형태
모든 $\lambda_i > 0$	단일 강한 최저점 (Strong Minimum)
모든 $\lambda_i < 0$	단일 강한 최대점 (Strong Maximum)
λ_i 중 양수와 음수가 모두 존재	안장점 (Saddle Point)
모든 $\lambda_i \geq 0$ 이고, 일부 $\lambda_j = 0$	약한 최저점 (Weak Minimum) 또는 정류점 없음
모든 $\lambda_i \leq 0$ 이고, 일부 $\lambda_j = 0$	약한 최대점 (Weak Maximum) 또는 정류점 없음

3.2 수치 최적화 1: 경사하강법(라인서치/스텝), 모멘텀

머신러닝 모델의 손실 함수는 대부분 복잡하고 비선형적이며, 해석적으로 최소값을 찾기 어렵습니다. 따라서 반복적인 수치 최적화 방법을 사용하는데, 그 중 가장 기본이 되는 것이 경사하강법입니다.

경사하강법 (Gradient Descent, GD)

직관적 이해: 경사하강법은 산에서 가장 빠르게 내려가는 길(가장 가파른 내리막길)을 찾아 한 걸음씩 내려가는 것과 같습니다. 이 '가장 가파른 내리막길'은 현재 위치에서의 그래디언트 벡터의 반대 방향입니다.

수학적 의미: 손실 함수 $L(\mathbf{w})$ 를 최소화하기 위해 모델 파라미터 $\mathbf{w}$ 를 업데이트하는 과정은 다음과 같습니다. $\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha \nabla L(\mathbf{w}_k)$ 여기서 $\mathbf{w}_k$ 는 k 번째 반복에서의 파라미터, $\nabla L(\mathbf{w}_k)$ 는 현재 파라미터에서의 그래디언트, α 는 **학습률(Learning Rate)**입니다.

학습률 (α):

- **역할:** 학습률은 한 번의 업데이트에서 파라미터가 그래디언트 방향으로 얼마나 크게 이동할지를 결정하는 하이퍼파라미터입니다.
- **안정성:**
 - **너무 큰 학습률:** 최적점을 건너뛰거나, 함수 값이 오히려 증가하여 발산할 수 있습니다. (Reference: The Laziest Programmer)
 - **너무 작은 학습률:** 수렴 속도가 매우 느려져 최적점에 도달하는 데 오랜 시간이 걸립니다.
- **라인 서치 (Line Search):** 매 반복마다 최적의 학습률 α 를 찾는 방법입니다. 이는 계산 비용이 많이 들 수 있지만, 더 안정적인 수렴을 보장합니다. 실제로는 고정된 학습률을 사용하거나, 학습률을 점진적으로 감소시키는 스케줄링 기법을 많이 사용합니다.

예제: 주어진 η 에서 발산/수렴 판단 함수 $f(x) = x^2$ 를 최소화하는 경사하강법을 생각해 봅시다. 그래디언트는 $\nabla f(x) = 2x$ 입니다. 업데이트 규칙은 $x_{k+1} = x_k - \alpha (2x_k) = (1 - 2\alpha)x_k$ 입니다.

- **수렴 조건:** $|1 - 2\alpha| < 1$ 이어야 합니다. 즉, $0 < \alpha < 1$ 일 때 수렴합니다.
- **발산 예시:** $\alpha = 1.1$ (너무 큰 학습률)인 경우, $x_{k+1} = (1 - 2 \times 1.1)x_k = (1 - 2.2)x_k = -1.2x_k$ 가 됩니다. 초기값 $x_0=1$ 이라면, $x_1=-1.2$, $x_2=1.44$, $x_3=-1.728$, ... 와 같이 값이 진동하며 발산합니다.

모멘텀 (Momentum)

직관적 이해: 모멘텀은 경사하강법에 '관성'을 부여하는 개념입니다. 공이 언덕을 굴러 내려올 때, 현재 기울기 뿐만 아니라 이전에 굴러온 속도(관성)도 고려하여 움직이는 것과 같습니다. 이는 최적화 과정에서 발생하는 불필요한 진동을 줄이고, 수렴 속도를 가속화하는 데 도움을 줍니다.

수학적 의미: 모멘텀을 적용한 경사하강법은 다음과 같이 두 단계로 업데이트됩니다. $\mathbf{v}_{k+1} = \beta \mathbf{v}_k + \nabla L(\mathbf{w}_k)$ $\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha \mathbf{v}_{k+1}$ 여기서 $\mathbf{v}$ 는 속도(velocity) 벡터로, 이전 그래디언트들의 지수 가중 평균을 나타냅니다. β 는 모멘텀 계수(보통 0.9 또는 0.99)로, 이전 속도의 영향을 얼마나 반영할지를 결정합니다. $\beta=0$ 이면 일반 경사하강법과 동일합니다.

이차 볼에서의 수렴 궤적 비교 (GD vs 모멘텀) 사례: 손실 함수 표면이 '계곡'처럼 한 방향으로만 가파르고 다른 방향으로만 완만한 '병적인 곡률(pathological curvature)'을 가질 때, 일반 경사하강법은 최적점을 향해 지그재그로 움직이며 수렴이 매우 느려집니다. (Reference: Distill.pub - zigzags toward the solution)

예를 들어, $L(x, y) = \frac{1}{16}x^2 + 9y^2$ 와 같은 타원형 등고선을 가진 함수에서, 경사하강법은 y 축 방향으로 심하게 진동하며 느리게 수렴합니다. 반면 모멘텀은 이러한 진동을 효과적으로 줄여주고, 계곡을 따라 더 빠르게 최적점으로 이동할 수 있도록 돕습니다. (Reference: Visualizing Gradient Descent with Momentum)

수렴률 및 조건수와 난이도

- **수렴률 (Convergence Rate):** 경사하강법은 이론적으로 지수적인 수렴률을 보이지만, 실제로는 손실 함수의 형태에 따라 수렴 속도가 크게 달라질 수 있습니다.
- **조건수 (Condition Number):** 손실 함수의 헤시안 행렬의 조건수($\kappa = \lambda_{\max} / \lambda_{\min}$)는 최적화 문제의 '난이도'를 나타냅니다. 조건수가 클수록 손실 함수 표면이 더 길고 좁은 계곡 형태를 띠게 되어, 경사하강법의 수렴이 느려집니다. (Reference: Distill.pub, Cornell CS4787) 모멘텀은 이러한 높은 조건수 문제에 대한 효과적인 해결책 중 하나입니다.

3.3 수치 최적화 2: 뉴턴법, 준뉴턴 (BFGS) 직관

경사하강법이 1차 미분 정보(그래디언트)만을 사용하는 반면, 뉴턴법은 2차 미분 정보(헤시안)까지 활용하여 더 효율적인 최적화를 시도합니다.

뉴턴법 (Newton's Method)

직관적 이해: 뉴턴법은 현재 위치에서 손실 함수를 2차 함수(포물선)로 근사한 다음, 이 근사 함수의 최저점으로 한 번에 이동하는 방식입니다. 마치 현재 위치 주변의 지형을 포물선으로 정확히 파악하고, 그 포물선의 가장 낮은 곳으로 점프하는 것과 같습니다.

수학적 의미: 함수 $f(\mathbf{w})$ 를 최소화하기 위해, 현재 파라미터 $\mathbf{w}_k$ 에서 $f(\mathbf{w})$ 를 2차 테일러 근사합니다. $f(\mathbf{w}) \approx f(\mathbf{w}_k) + \nabla f(\mathbf{w}_k)^T (\mathbf{w} - \mathbf{w}_k) + \frac{1}{2} (\mathbf{w} - \mathbf{w}_k)^T H_f(\mathbf{w}_k) (\mathbf{w} - \mathbf{w}_k)$ 이 2차 근사 함수의 최소값을 찾기 위해 미분하여 0으로 두면, 다음 업데이트 규칙을 얻습니다. $\mathbf{w}_{k+1} = \mathbf{w}_k - H_f(\mathbf{w}_k)^{-1} \nabla f(\mathbf{w}_k)$ 여기서 $H_f(\mathbf{w}_k)$ 는 현재 파라미터에서의 헤시안 행렬입니다.

동일 데이터에서 GD vs 뉴턴 한 걸음 비교 사례: 아래 그림은 경사하강법(녹색)과 뉴턴법(빨간색)이 함수를 최소화하는 궤적을 비교한 것입니다. 뉴턴법은 함수의 곡률 정보를 활용하기 때문에, 경사하강법이 지그재그로 여러 번 이동해야 하는 경로를 단 몇 걸음 만에 최적점으로 수렴시킬 수 있습니다. 특히 2차 함수에 대해서는 단 한 번의 스텝으로 최적점에 도달합니다. (Reference: Newton's method in optimization - comparison image, Cornell CS4780 - Newton's Method is much faster)

장점:

- **빠른 수렴:** 2차 수렴(Quadratic Convergence) 특성을 가지므로, 최적점 근처에서는 매우 빠르게 수렴합니다. (Reference: Meegle, Medium)
- **학습률 튜닝 불필요:** 별도의 학습률 하이퍼파라미터가 필요 없습니다. (Reference: Baeldung)

단점:

- **높은 계산 비용:**
 - 헤시안 행렬 $H_f(\mathbf{w}_k)$ 를 계산하는 데 $O(N^2)$ 의 비용이 듭니다 (N은 파라미터 수).
 - 헤시안 행렬의 역행렬 $H_f(\mathbf{w}_k)^{-1}$ 을 계산하는 데 $O(N^3)$ 의 비용이 듭니다. (Reference: Meegle, Medium)
 - 파라미터 수가 수백만 개에 달하는 딥러닝 모델에서는 이러한 계산이 현실적으로 불가능합니다.

- **메모리 요구량:** 헤시안 행렬을 저장하는 데 $O(N^2)$ 의 메모리가 필요합니다. (Reference: Medium)
- **발산 가능성:** 헤시안 행렬이 양의 정부호가 아닐 경우(예: 안장점이나 극대점 근처) 뉴턴법은 발산하거나 잘못된 방향으로 수렴할 수 있습니다. (Reference: Cornell CS4780)
- **전체 데이터셋 필요:** 헤시안 계산을 위해 일반적으로 전체 데이터셋의 그래디언트 정보가 필요하여, 미니배치 학습에 적용하기 어렵습니다. (Reference: Medium)

예제: 뉴턴 업데이트 1회 계산 함수 $f(x) = x^2 - 4x + 5$ 를 최소화하는 뉴턴법을 생각해 봅시다. 초기값 $x_0 = 0$ 에서 1회 업데이트를 계산합니다.

1. **1차 미분 (그래디언트):** $\nabla f(x) = f'(x) = 2x - 4$
2. **2차 미분 (헤시안):** $H_f(x) = f''(x) = 2$
3. **초기값 $x_0 = 0$ 에서의 값:**
 - $\nabla f(0) = 2(0) - 4 = -4$
 - $H_f(0) = 2$
4. **뉴턴 업데이트:** $x_1 = x_0 - H_f(x_0)^{-1} \nabla f(x_0) = 0 - (2)^{-1} (-4) = 0 - \frac{1}{2}(-4) = 0 + 2 = 2$
 실제 함수 $f(x) = (x-2)^2 + 1$ 의 최소점은 $x=2$ 입니다. 뉴턴법은 2차 함수이므로 단 한 번의 스텝으로 정확한 최소점을 찾았습니다.

준뉴턴법 (Quasi-Newton Methods)

직관적 이해: 준뉴턴법은 뉴턴법의 빠른 수렴 속도를 유지하면서도, 헤시안 행렬을 직접 계산하고 역행렬을 구하는 높은 비용을 피하기 위해 '헤시안을 근사'하는 방법입니다. 헤시안을 직접 계산하는 대신, 그래디언트의 변화를 통해 헤시안의 역행렬을 점진적으로 업데이트하여 근사합니다.

헤시안 활용/근사헤시안/계산비용 트레이드오프: 뉴턴법의 가장 큰 단점은 헤시안 계산 및 역행렬 계산 비용입니다. 준뉴턴법은 이 문제를 해결하기 위해 다음과 같은 트레이드오프를 가집니다.

- **헤시안 근사:** 헤시안 행렬 $H_f(\mathbf{w}_k)$ 또는 그 역행렬 $H_f(\mathbf{w}_k)^{-1}$ 을 직접 계산하는 대신, 이전 반복에서 얻은 그래디언트 정보를 활용하여 근사 행렬을 구축합니다.
- **업데이트:** 매 반복마다 근사 행렬을 효율적으로 업데이트합니다. 이는 $O(N^2)$ 의 비용으로 가능하며, 뉴턴법의 $O(N^3)$ 보다 훨씬 효율적입니다.

BFGS (Broyden-Fletcher-Goldfarb-Shanno): BFGS는 가장 널리 사용되는 준뉴턴법 중 하나입니다. 헤시안의 역행렬을 직접 근사하여 업데이트 규칙을 다음과 같이 변형합니다. $\mathbf{w}_{k+1} = \mathbf{w}_k - B_k \nabla f(\mathbf{w}_k)$ 여기서 B_k 는 헤시안 역행렬의 근사 행렬이며, 매 반복마다 그래디언트의 변화를 이용하여 업데이트됩니다. BFGS는 뉴턴법에 버금가는 빠른 수렴 속도를 보이면서도 계산 비용이 훨씬 낮아, 머신러닝 및 최적화 분야에서 광범위하게 활용됩니다. L-BFGS(Limited-memory BFGS)는 BFGS의 메모리 사용량을 줄인 버전으로, 대규모 문제에 특히 유용합니다.

결론적으로, 경사하강법은 구현이 간단하고 대규모 데이터에 적용하기 용이하여 딥러닝에서 가장 널리 사용되지만, 수렴 속도가 느릴 수 있습니다. 모멘텀은 이러한 경사하강법의 단점을 보완하여 수렴을 가속화합니다. 뉴턴법은 이론적으로 가장 빠른 수렴을 보장하지만, 높은 계산 비용으로 인해 대규모 머신러닝 모델에는 직접 적용하기 어렵습니다. 준뉴턴법(BFGS 등)은 뉴턴법의 장점을 취하면서 계산 비용을 줄인 절충안으로, 특정 최적화 문제에서 효과적으로 사용될 수 있습니다.